LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Adinda Olivia Maharani 2609106088

Kelas C1 ‘26

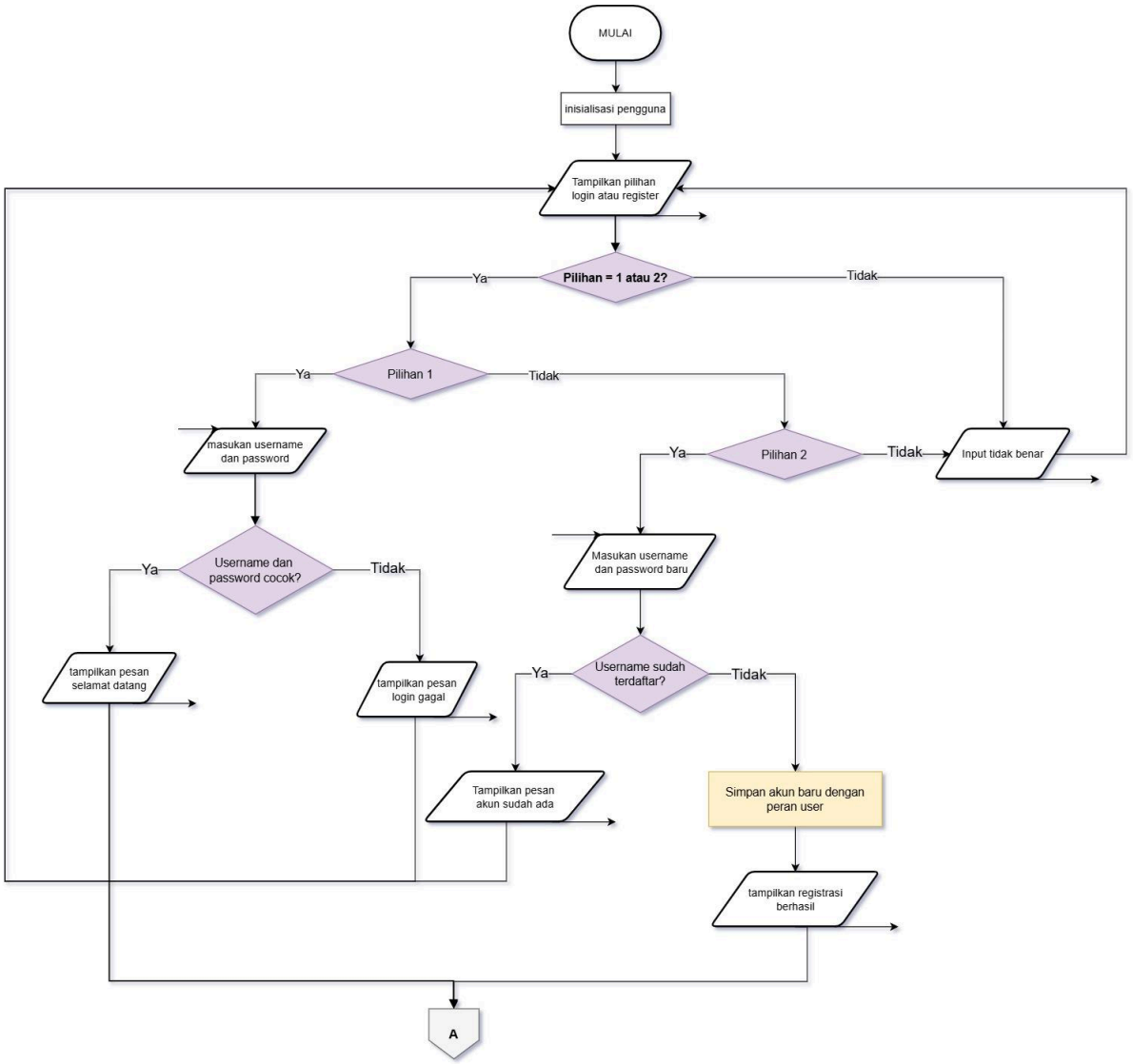
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

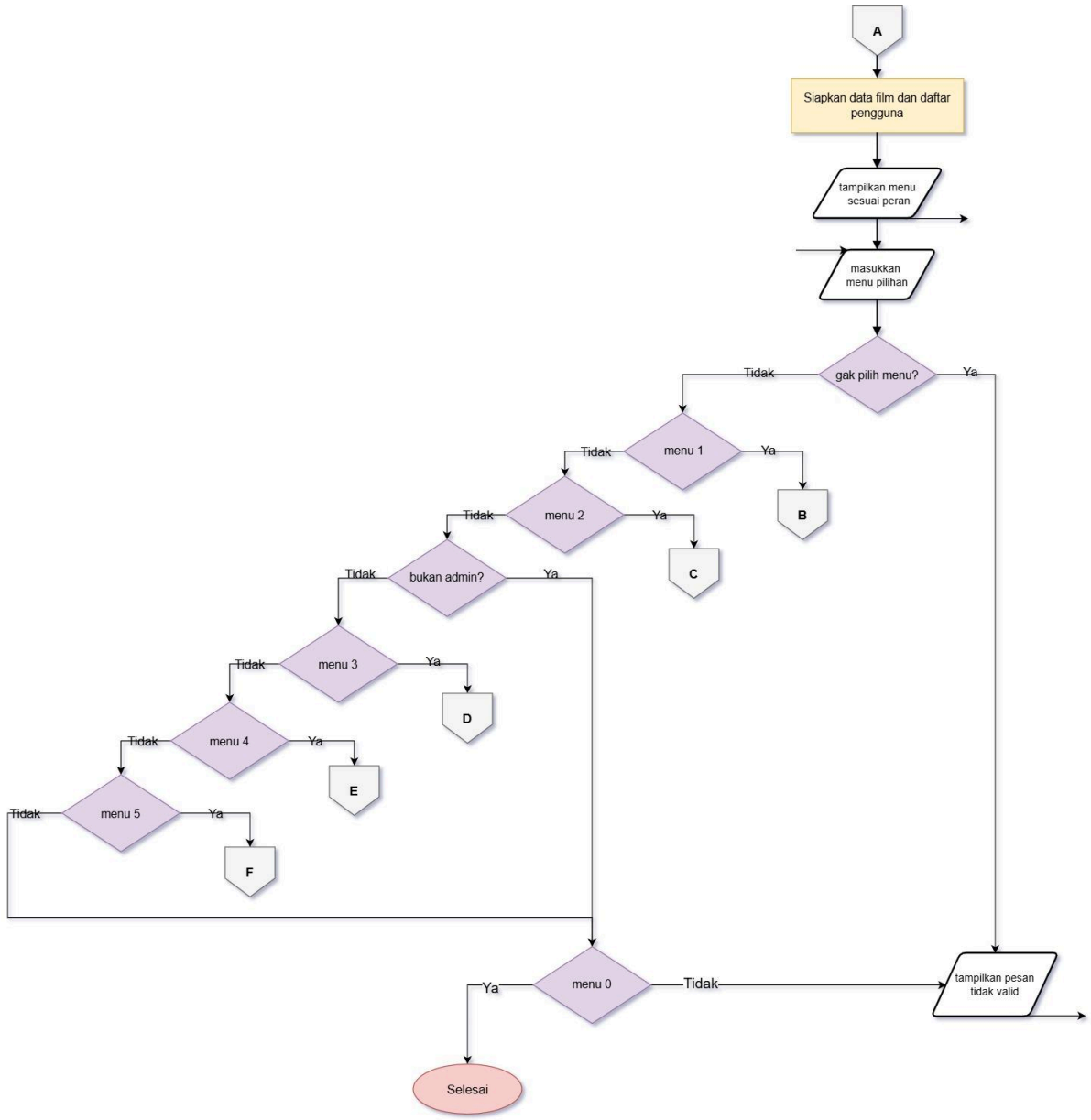
SAMARINDA

2026

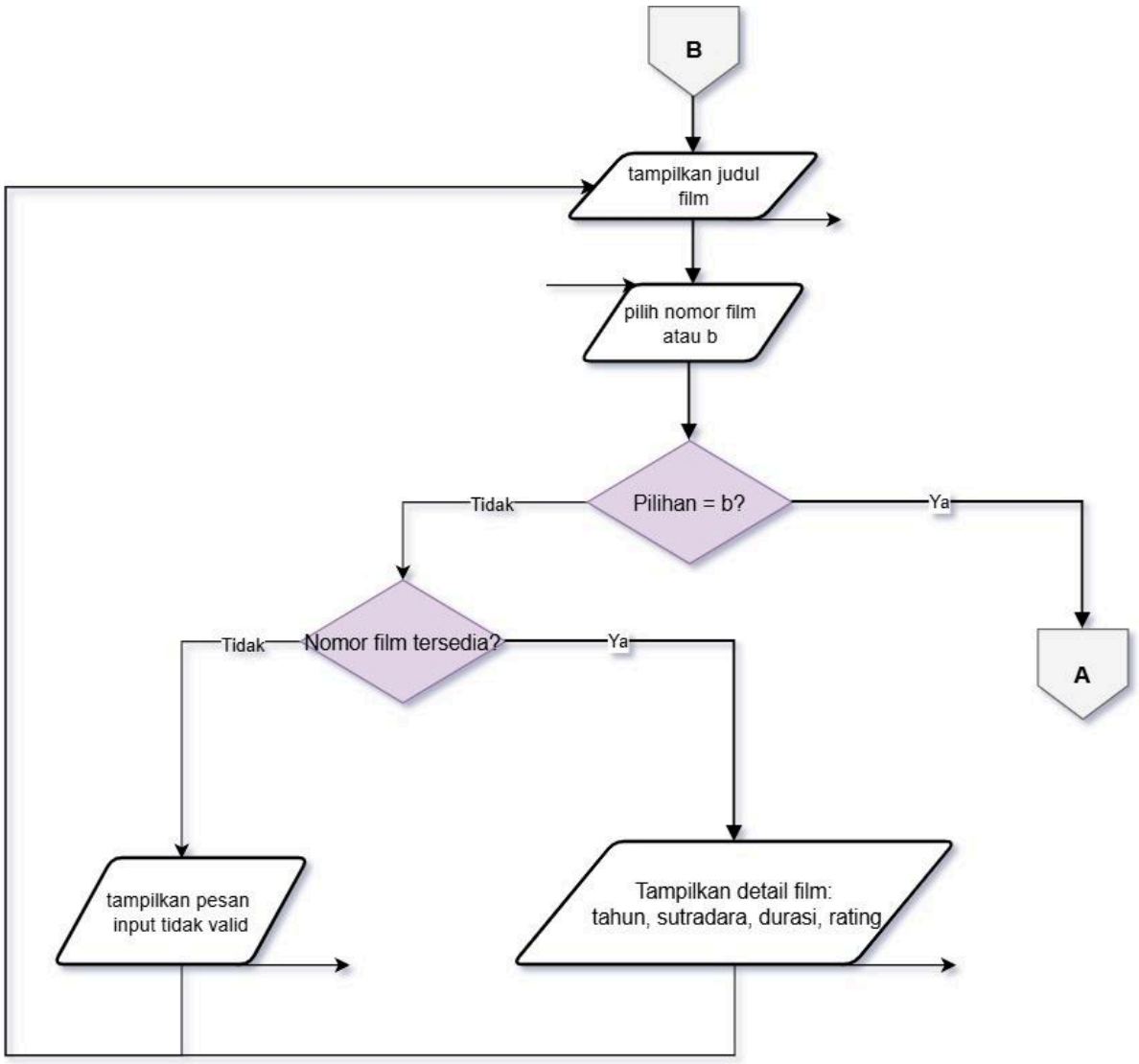
1. Flowchart



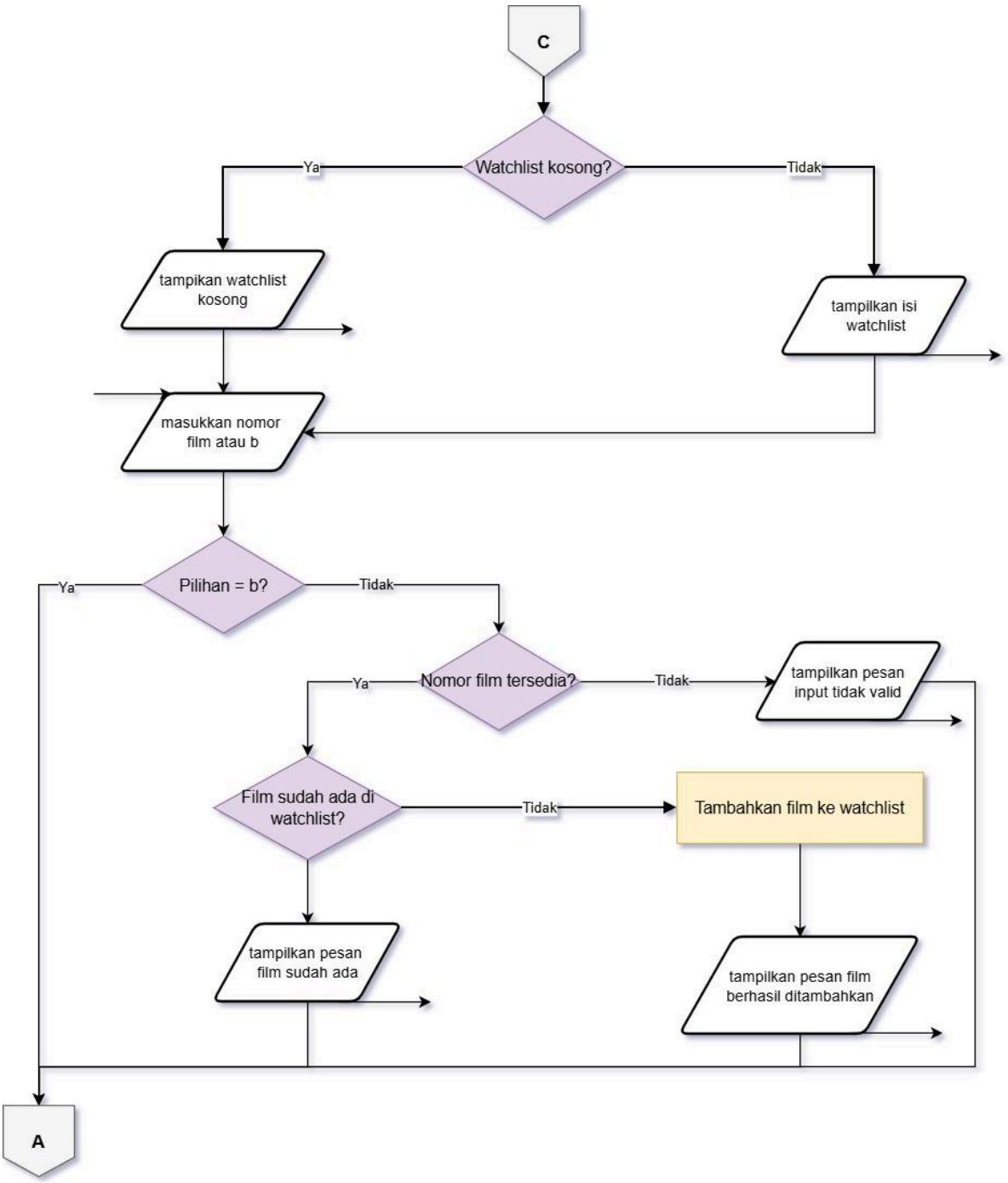
Gambar 1.1


A
Siapkan data film dan daftar pengguna
tampilkan menu sesuai peran
masukkan menu pilihan
gak pilih menu?
Tidak
Ya
menu 1
Tidak
Ya
B
menu 2
Tidak
Ya
C
bukan admin?
Tidak
Ya
menu 3
Tidak
Ya
D
menu 4
Tidak
Ya
E
menu 5
Tidak
Ya
F
menu 0
Ya
Tidak
tampilkan pesan tidak valid
Selesai

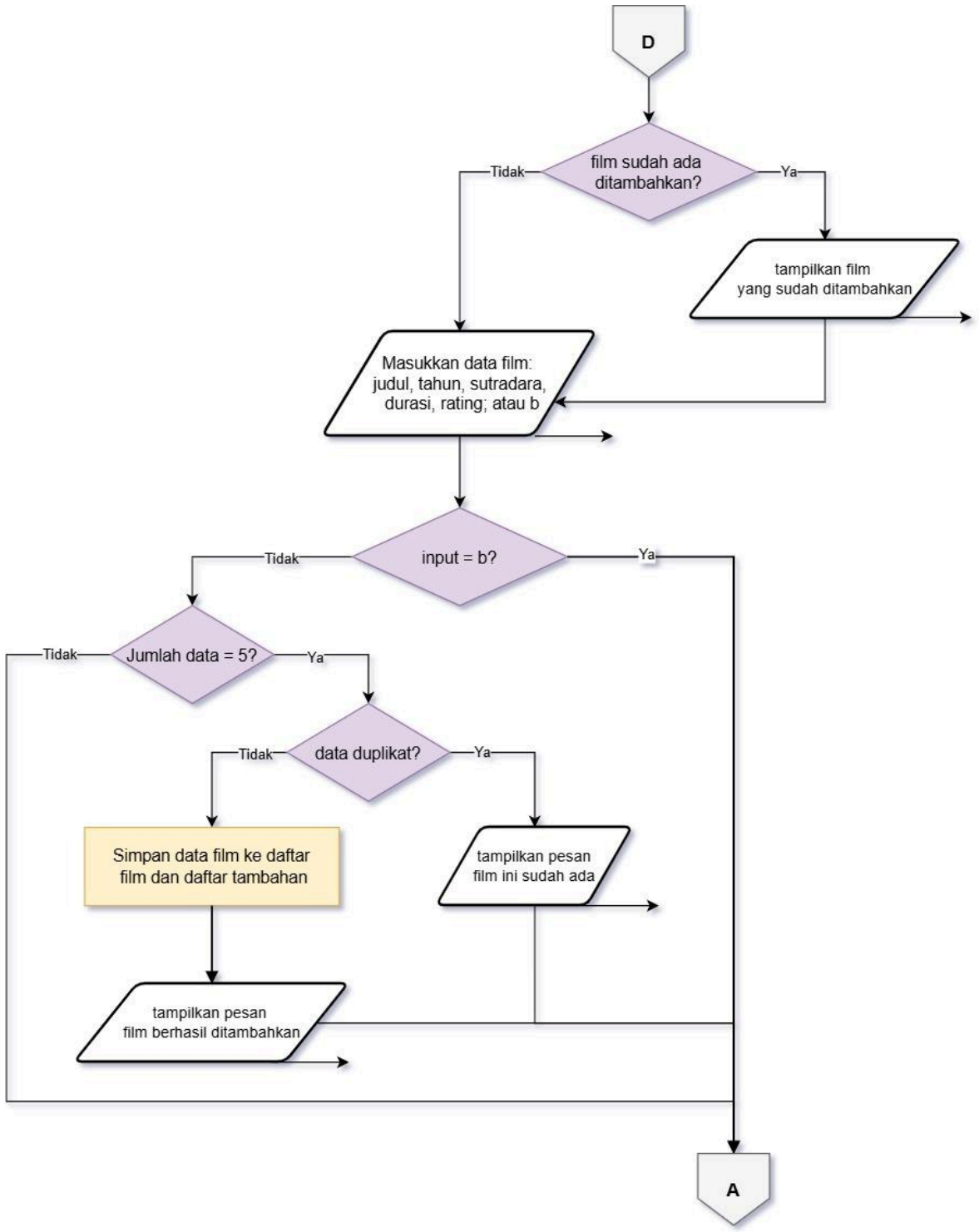
Gambar 2.1


B
tampilkan judul film
pilih nomor film atau b
Pilihan = b?
Tidak
Ya
Nomor film tersedia?
Tidak
Ya
A
tampilkan pesan input tidak valid
Tampilkan detail film: tahun, sutradara, durasi, rating

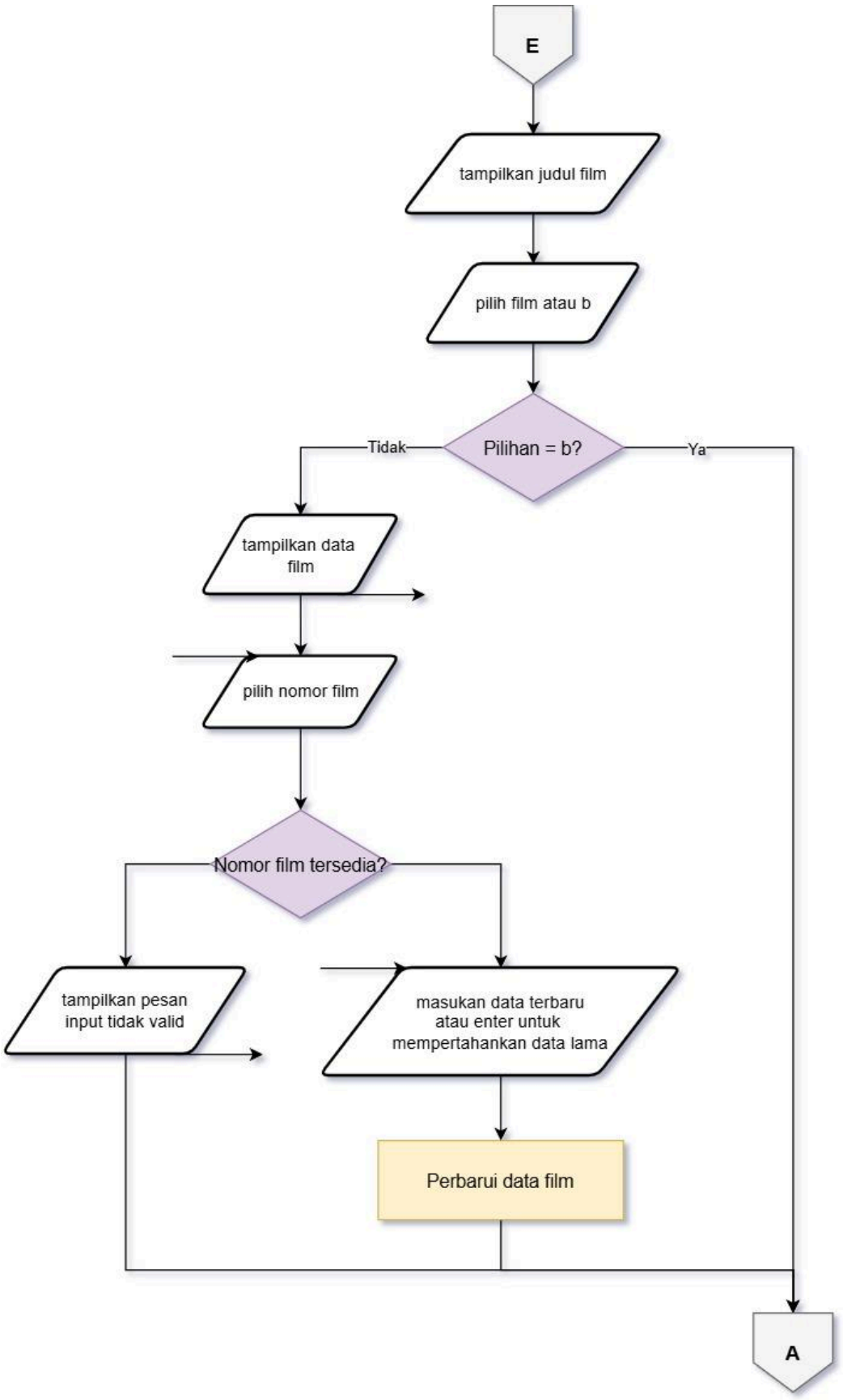
Gambar 3.1


C
Watchlist kosong?
Ya
Tidak
tampikan watchlist kosong
tampilkan isi watchlist
masukkan nomor film atau b
Pilihan = b?
Ya
Tidak
Nomor film tersedia?
Ya
Tidak
tampilkan pesan input tidak valid
Film sudah ada di watchlist?
Tidak
Tambahkan film ke watchlist
tampilkan pesan film sudah ada
tampilkan pesan film berhasil ditambahkan
A

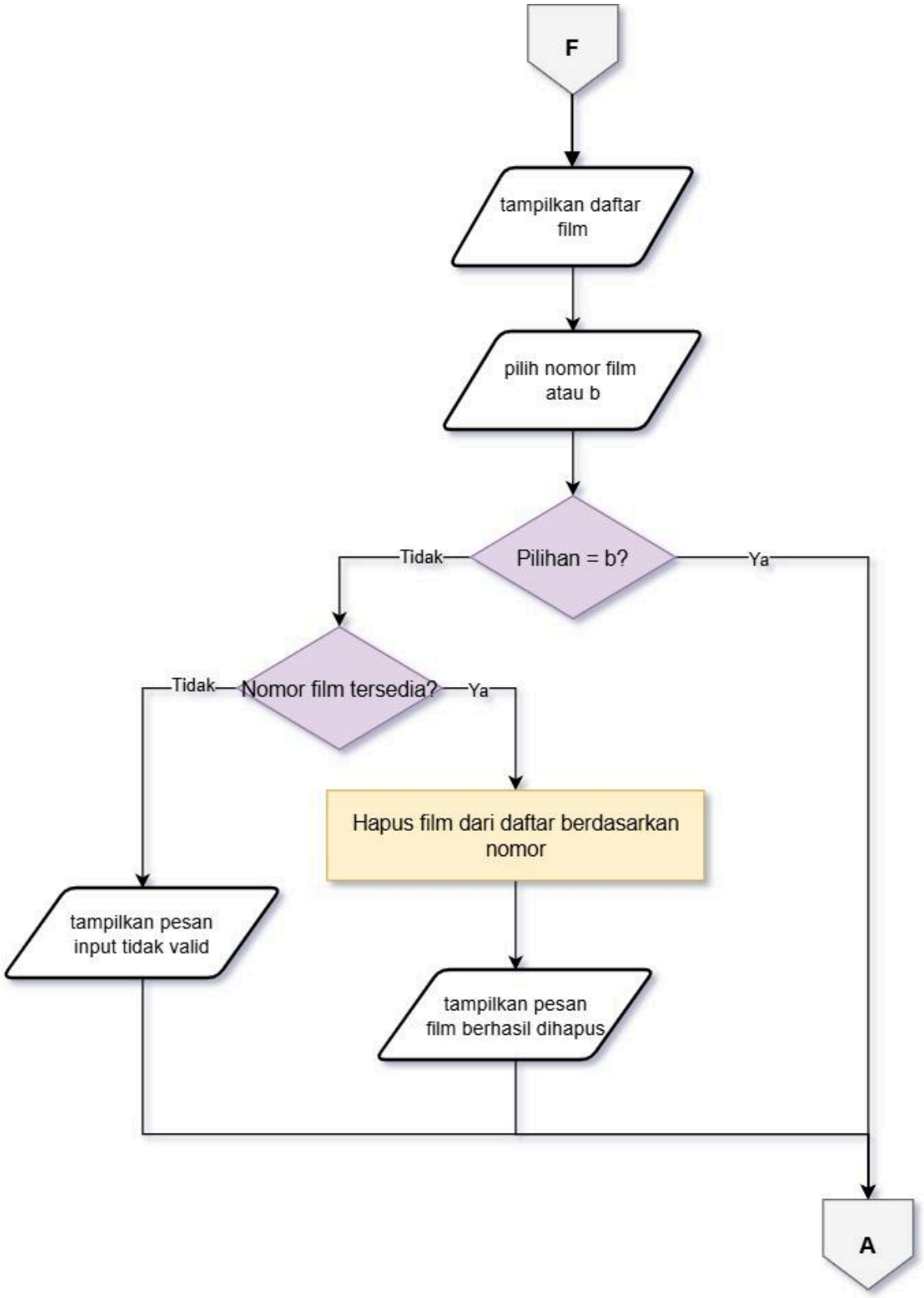
Gambar 4.1


D
film sudah ada ditambahkan?
Tidak
Ya
tampilkan film yang sudah ditambahkan
Masukkan data film: judul, tahun, sutradara, durasi, rating; atau b
input = b?
Tidak
Ya
Jumlah data = 5?
Tidak
Ya
data duplikat?
Tidak
Ya
Simpan data film ke daftar film dan daftar tambahan
tampilkan pesan film ini sudah ada
tampilkan pesan film berhasil ditambahkan
A

Gambar 5.1


E
tampilkan judul film
pilih film atau b
Pilihan = b?
Tidak
Ya
tampilkan data film
pilih nomor film
Nomor film tersedia?
tampilkan pesan input tidak valid
masukan data terbaru atau enter untuk mempertahankan data lama
Perbarui data film
A

Gambar 6.1


F
tampilkan daftar film
pilih nomor film atau b
Pilihan = b?
Tidak
Ya
Nomor film tersedia?
Tidak
Ya
Hapus film dari daftar berdasarkan nomor
tampilkan pesan input tidak valid
tampilkan pesan film berhasil dihapus
A

Gambar 7.1

2. Deskripsi Singkat Program

Sistem Manajemen Film ini berbasis Python yang dirancang untuk mengelola pustaka film dan daftar tontonan pengguna melalui Command-Line Interface (CLI). Program ini dilengkapi dengan mekanisme autentikasi dan pembatasan hak akses berbasis peran, sehingga membedakan alur kerja antara pengguna biasa dan admin.

3. Source Code

```
import os

users = {
    "ovoxo": ["rahasia", "admin"],
    "milo": ["itu", "user"],
}
list_films = [
    {"title": "Whiplash", "release": "2014", "director": "Damien Chazelle",
"duration": "107 minutes", "rating": "4.4"},
    {"title": "Interstellar", "release": "2014", "director": "Christopher
Nolan", "duration": "169 minutes", "rating": "4.5"},
    {"title": "Taxi driver", "release": "1976", "director": "Martin Scorsese",
"duration": "114 minutes", "rating": "4.1"},
    {"title": "The godfather", "release": "1972", "director": "Francis Ford
Coppola", "duration": "175 minutes", "rating": "4.5"},
]
watchlist_user = []
current_user = None

def detail_films(index):
    film = list_films[index]
    print(f"\n--- {film['title']} ---")
    print(f"release: {film['release']} | director: {film['director']} |
duration: {film['duration']} | rating: {film['rating']}\n")

def updated_films(films, index=0):
    if index >= len(films):
        return
    print(f"{index + 1}. {films[index]['title']}")
    updated_films(films, index + 1)

def get_user_role():
```

```
    if current_user in users:
        return users[current_user][1]
    return "user"

def main():
    global current_user, watchlist_user, list_films
```

```
 username = input("Enter username: ")
    password = input("Enter password: ")
    if username in users and password == users[username][0]:
        current_user = username
        print(f"\nWelcome {username}!\n")
    else:
        print("Invalid credentials.")
        return

    while True:
        user_role = get_user_role()
        print("[1] list films  [2] watchlist")
        if user_role == "admin":
            print("[3] add films   [4] edit data films   [5] delete film")
        print("[0] exit")

        choices = input("Choose option: ")
```

```
  if choices == "1":
            updated_films(list_films)
            input_index = input("Choose index to see detail (b to cancel):
")
            if input_index.isdigit() and 1 <= int(input_index) <=
len(list_films):
                detail_films(int(input_index) - 1)
```

```
 elif choices == "2":
            print("--- Your Watchlist ---", watchlist_user)
            input_index = input("Add film index to watchlist (b to cancel): ")
            if input_index.isdigit() and 1 <= int(input_index) <=
len(list_films):
                title_film = list_films[int(input_index) - 1]['title']
                if title_film not in watchlist_user:
                    watchlist_user.append(title_film)
                    print(f"{title_film} added to watchlist!")
```

```
elif choices == "3" and user_role == "admin":
            input_film = input("Enter: title, release, director, duration,
rating:\n")
            film_data = [item.strip() for item in input_film.split(",")]
            if len(film_data) == 5:
                list_films.append({
                    "title": film_data[0], "release": film_data[1],
                    "director": film_data[2], "duration": film_data[3],
"rating": film_data[4]
                })
                print("Successfully added!")
```

```
elif choices == "4" and user_role == "admin":
            updated_films(list_films)
            input_index = input("Pick an index film to change (b to back): ")
            if input_index.isdigit() and 1 <= int(input_index) <=
len(list_films):
                idx = int(input_index) - 1
                detail_films(idx)
                f = list_films[idx]
                list_films[idx] = {
                    "title": input(f"title [{f['title']}]: ") or f['title'],
                    "release": input(f"release [{f['release']}]: ") or
f['release'],
                    "director": input(f"directed by [{f['director']}]: ") or
f['director'],
                    "duration": input(f"duration [{f['duration']}]: ") or
f['duration'],
                    "rating": input(f"ratings [{f['rating']}]: ") or f['rating']
                }
                print("Data successfully updated!")
```

```
elif choices == "5" and user_role == "admin":
            updated_films(list_films)
            input_index = input("Pick an index film to delete (b to back): ")
            if input_index.isdigit() and 1 <= int(input_index) <=
len(list_films):
                deleted = list_films.pop(int(input_index) - 1)
                print(f"Film {deleted['title']} successfully deleted.")

        elif choices == "0":
            break

if __name__ == "__main__":
    main()
```

A. Fitur Create

Menambahkan User Baru (Register)

Pendaftaran akun menyimpan username, password, dan role baru ke dalam dictionary users.

```
users[new_username] = [new_password, "user"]
```

Kode ini menambahkan pasangan key-value baru ke dictionary users, di mana new_username menjadi key, dan list berisi [new_password, "user"] menjadi value-nya.

Menambahkan Film Baru oleh Admin (Opsi 3)

Admin memasukkan data film baru yang kemudian ditambahkan ke dalam list_films dan added_films.

```
added_films.append(film_data[0])
list_films.append({
    "title": film_data[0],
    "release": film_data[1],
    "director": film_data[2],
    "duration": film_data[3],
    "rating": film_data[4]
})
```

.append() menambah dictionary berisi detail film baru (judul, tahun rilis, sutradara, durasi, dan rating) ke dalam daftar utama list_films.

Menambahkan Film ke Watchlist User (Opsi 2)

Pengguna memilih film dari daftar utama untuk dimasukkan ke daftar tonton pribadi (watchlist_user).

```
if title_film not in watchlist_user:
    watchlist_user.append(title_film)
```

Setelah memvalidasi bahwa film belum ada di dalam daftar, metode .append() menambah judul film tersebut ke dalam list watchlist_user.

B. Fitur Read

Menampilkan Daftar Judul Film (Rekursif)

Fungsi updated_films membaca dan mencetak seluruh judul film dari list secara berurutan.

```
def updated_films(films, index=0):
    if index >= len(films):
        return
    print(f"{index + 1}. {films[index]['title']}")
    updated_films(films, index + 1)
```

Fungsi ini mengakses elemen films[index] untuk membaca nilai atribut 'title' dan memanggil dirinya sendiri secara rekursif hingga seluruh isi list ditampilkan.

Menampilkan Detail Informasi Film

Fungsi detail_films membaca seluruh properti dari satu film spesifik.

```
def detail_films(index):
    try:
        film = list_films[index]
        print(f"\n--- {film['title']} ---")
        print(f"release: {film['release']}")
        print(f"directed by: {film['director']}")
        print(f"duration: {film['duration']}")
        print(f"ratings: {film['rating']}\n")
    except IndexError:
        print("invalid input.")
```

Kode membaca item pada indeks tertentu di list_films dan mengambil key 'title', 'release', 'director', 'duration', dan 'rating' untuk dicetak ke layar.

Menampilkan Watchlist Pengguna

Membaca dan menampilkan item yang tersimpan di dalam list watchlist.

```
for i, film in enumerate(watchlist_user):
    print(f"{i+1}. {film}")
```

Menggunakan perulangan for dan enumerate() untuk membaca setiap string judul film di dalam watchlist_user beserta nomor urutnya.

C. Fitur Update

Mengedit Data Film oleh Admin (Opsi 4)

Admin memilih film berdasarkan indeks, lalu dapat memperbarui informasinya atau mempertahankan data lama.

```
list_films[idx] = {
    "title": title,
    "release": release,
    "director": directed,
    "duration": duration,
    "rating": ratings
}
```

Kode ini menimpa (overwrite) elemen list_films pada posisi indeks idx dengan dictionary baru yang berisi data hasil perbaikan/input pengguna.

D. Fitur Delete

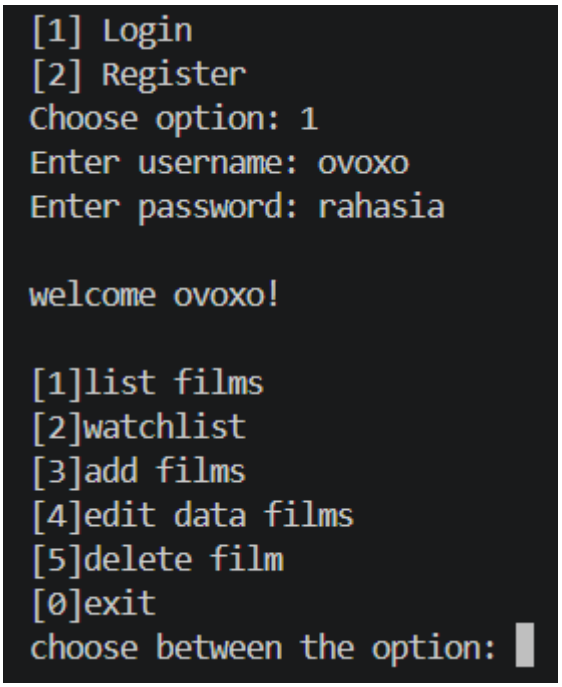
Menghapus Film dari List Utama oleh Admin (Opsi 5)

Admin menghapus salah satu film dari daftar list_films.

```
deleted_film = list_films.pop(idx)
```

.pop(idx) menghapus elemen pada posisi indeks idx dari list_films sekaligus mengembalikan (return) data elemen yang dihapus tersebut.

4. Hasil Output



Gambar 13.1 (Tampilan menu pada admin)

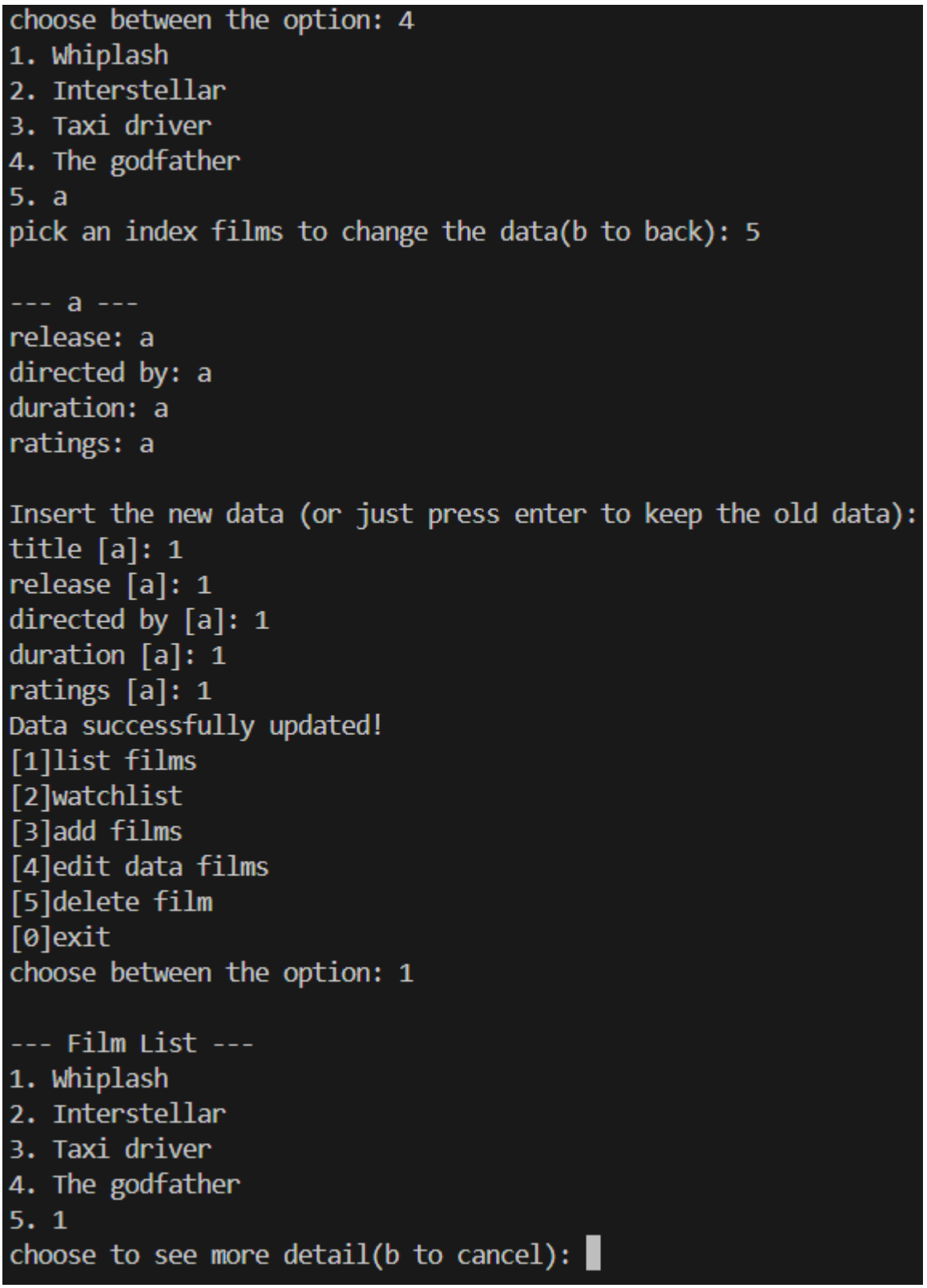
```
choose between the option: 3
you haven't added film yet
add your films include the data here
for example: (title film), (years release), (director), (film duration), (rating)
(b to cancel):a,a,a,a,a
successfully added!
[1]list films
[2]watchlist
[3]add films
[4]edit data films
[5]delete film
[0]exit
choose between the option: 1

--- Film List ---
1. Whiplash
2. Interstellar
3. Taxi driver
4. The godfather
5. a
choose to see more detail(b to cancel): 5

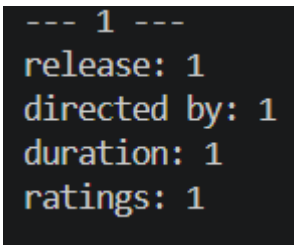
--- a ---
release: a
directed by: a
duration: a
ratings: a


--- Film List ---
1. Whiplash
2. Interstellar
3. Taxi driver
4. The godfather
5. a
choose to see more detail(b to cancel):
```

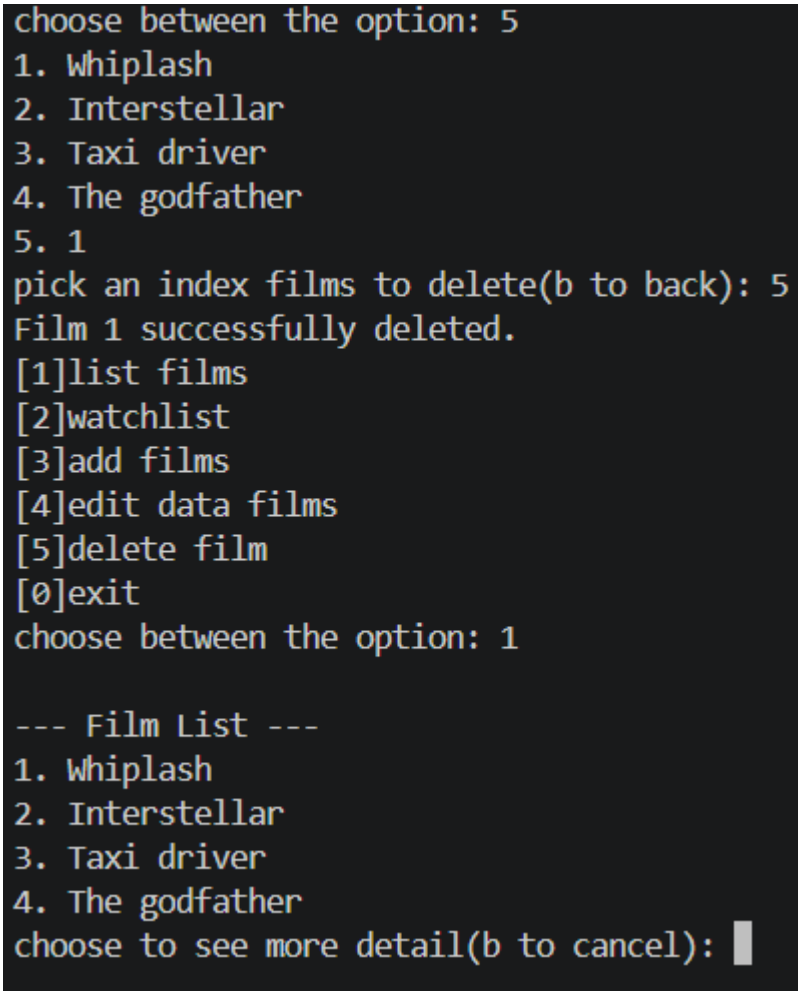
Gambar 14.1 (Fitur menambahkan film beserta isi data nya)



Gambar 15.1 (Ftiur mengedit data film tersebut)



Gambar 16.1 (Bukti bahwa data film berhasil diubah)



Gambar 16.2 (Fitur menghapus film)

```
[1] Login
[2] Register
Choose option: 2
Enter new username: 1
Enter new password: 1
Registration successful! Please login.

[1] Login
[2] Register
Choose option: 1
Enter username: 1
Enter password: 1

welcome 1!

[1]list films
[2]watchlist
[0]exit
choose between the option:
```

Gambar 17.1 (Membuat akun baru dan tampilan menu pada pengguna biasa)

```
choose between the option: 2
your watchlist is empty!
add film index here(b to cancel): 4
The godfather successfully added to your watchlist!
[1]list films
[2]watchlist
[0]exit
choose between the option: 2
--- Your Watchlist ---
1. The godfather
add film index here(b to cancel):
```

Gambar 17.2 (Fitur menambahkan watchlist)

```
--- Film List ---
1. Whiplash
2. Interstellar
3. Taxi driver
4. The godfather
choose to see more detail(b to cancel): 4

--- The godfather ---
release: 1972
directed by: Francis Ford Coppola
duration: 175 minutes
ratings: 4.5


--- Film List ---
1. Whiplash
2. Interstellar
3. Taxi driver
4. The godfather
choose to see more detail(b to cancel): 3

--- Taxi driver ---
release: 1976
directed by: Martin Scorsese
duration: 114 minutes
ratings: 4.1
```

Gambar 18.2 (Fitur menampilkan daftar film beserta data-nya)

5. Langkah-langkah Git

5. 1 Git Add

```
PS C:\praktikum_apd_2026> git add .
```

Perintah git add . digunakan untuk menandai file program baru yang ada di dalam folder posttest-apd-7 agar siap disimpan oleh Git.

5.2 Git Commit

```
PS C:\praktikum_apd_2026> git commit -m "pdf otw"
[main fde4a59] pdf otw
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 kelas/u.py
```

Perintah git commit -m “pdf nya nyusul” digunakan untuk menyimpan file program 2609106088-ADINDAOLIVIAMAHARANI-PT-7 ke dalam riwayat Git yang sudah ada lengkap beserta catatan perubahannya.

5.3 Git Push

```
PS C:\praktikum_apd_2026> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.30 KiB | 111.00 KiB/s, done.
Total 7 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/adindaoliviaa2-spec/praktikum_apd_2026.git
   f2e7489..fde4a59  main -> main
```

Perintah git push digunakan untuk mengupload file program yang sudah disimpan tadi sebelumnya dari laptop ke tempat penyimpanan online di GitHub.